Performance Measurement(A+B)

Date: 2026-3-15

Declaration

I hereby declare that all the work done in this project titled "Performance Measurement(A+B)" is of my independent effort.

Chapter 1: Introduction

1.1 Description

Given S as a collection of N positive integers that are no more than V. For any given number c , you are supposed to find two integers a and b from S,so that $a + b = c$.

Here follows two algorithms for the problem.

1.2.1 Algorithm 1:Brute-force Algorithm.

This algorithm checks all possible pairs of elements in the array to see if their sum equals the target. It uses two nested loops: the outer loop iterates over each element as the first candidate, and the inner loop checks the subsequent elements as the second candidate. If a pair (arr[i], arr[j]) with i < j satisfies arr[i] + arr[j] == target, the algorithm returns the pair. Otherwise, it returns 0 after examining all combinations.

Time Complexity: $O(N^2)$(due to the double loop)

1.2.2 Algorithm 2:Sort + Two integers

This algorithm first sorts the array using qsort (O(N log N)). After sorting, it employs a two-pointer technique: a left pointer starts at the beginning (smallest element) and a right pointer at the end (largest element). At each step, it computes the sum of the elements pointed to. If the sum equals the target, the pair is found. If the sum is less than the target, the left pointer is moved right to increase the sum; if greater, the right pointer is moved left to decrease the sum. This process continues

until the pointers meet or failed.

Time Complexity:O(N log N)(due to the sorting)

Chapter 2: Algorithm Specification

2.1Algorithm BruteForce(S, N, c)

Input: Array S of N integers, target sum c
Output: (true, a, b) if a+b=c found, else (false, 0, 0)

```
for i =0 to N-1:
    for j = i+1 to N-1:
        if S[i] + S[j] = c then
            return (true, S[i], S[j])
        end if
    end for
end for
return (false, 0, 0)
```

2.2Algorithm TwoPointer(S, N, c)

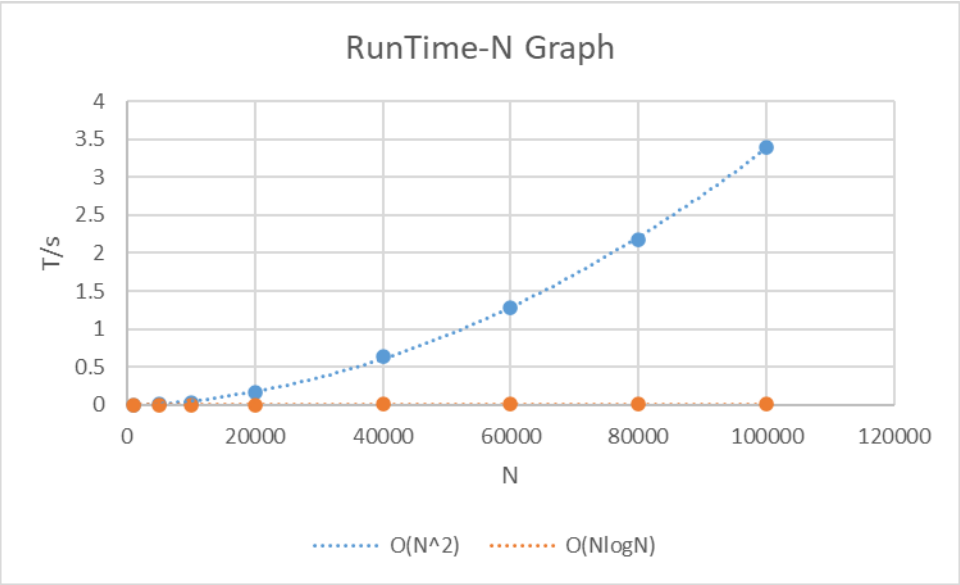
Input: Array S of N integers, target sum c
Output: (true, a, b) if a+b=c found, else (false, 0, 0)

```
// Sort the temporary array
sort(temp, N)
left =0
right = N-1
while left < right do
    sum = temp[left] + temp[right]
    if sum = c then
        return (true, temp[left], temp[right])
    else if sum < c then
        left = left + 1
    else
        right = right - 1
    end if
end while

return (false, 0, 0)
```

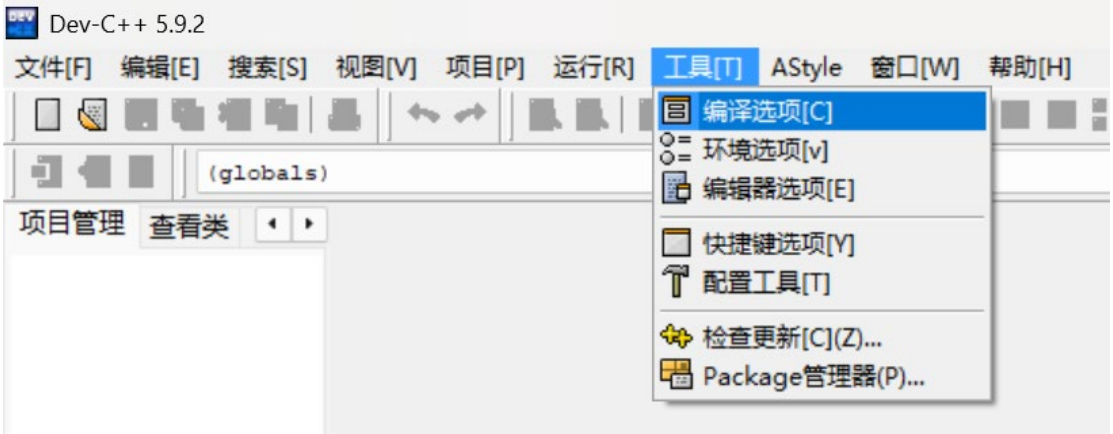
Chapter 3: Testing Results

V=10*N	N	1000	5000	10000	20000	40000	60000	80000	100000
Algorithm 1	Iterations (K)	1000	1000	1000	10	10	10	10	10
	Ticks	409	8568	35573	1581	6432	12884	21797	33993
	Total Time(sec)	0.409000	8.568000	35.573000	1.581000	6.432000	12.884000	21.797000	33.993000
	Duration(sec)	0.000409	0.008568	0.035573	0.158100	0.643200	1.288400	2.179700	3.399300
Algorithm 2	Iterations (K)	1000	1000	1000	10	10	10	10	10
	Ticks	49	267	739	16	37	49	43	85
	Total Time(sec)	0.049000	0.267000	0.739000	0.016000	0.037000	0.049000	0.043000	0.085000
	Duration(sec)	0.000049	0.000267	0.000739	0.001600	0.003700	0.004900	0.004300	0.008500



Note: Due to the slight difference of specific computer and other factors, the data may be a bit difference as in the table.

If you cann't run the program on a Dev-C++,please select the tool option ,compiler suboption,and add the following command: **-std=c99**


Dev-C++ 5.9.2
文件[F] 编辑[E] 搜索[S] 视图[V] 项目[P] 运行[R] 工具[T] AStyle 窗口[W] 帮助[H]
(globals)
项目管理 查看类
编译选项[C]
环境选项[v]
编辑器选项[E]
快捷键选项[Y]
配置工具[T]
检查更新[C](Z)...
Package管理器(P)...

编译器选项
设定编译器配置
TDM-GCC 4.8.1 64-bit Release
编译器 代码生成/优化 目录 程序
编译时加入以下命令:
-std=c99
在连接器命令行加入以下命令
-static-libgcc

Here are two typical results:

```
368 0.368000 0.000368
31 0.031000 0.000031
8282 8.282000 0.008282
273 0.273000 0.000273
33019 33.019000 0.033019
596 0.596000 0.000596
1333 1.333000 0.133300
10 0.010000 0.001000
5473 5.473000 0.547300
33 0.033000 0.003300
12567 12.567000 1.256700
45 0.045000 0.004500
22860 22.860000 2.286000
59 0.059000 0.005900
42487 42.487000 4.248700
87 0.087000 0.008700
32767
--------------------------
Process exited after 128.
请按任意键继续. . .
```

Dev

```
409 0.409000 0.000409
49 0.049000 0.000049
8568 8.568000 0.008568
267 0.267000 0.000267
35573 35.573000 0.035573
739 0.739000 0.000739
1581 1.581000 0.158100
16 0.016000 0.001600
6432 6.432000 0.643200
37 0.037000 0.003700
12884 12.884000 1.288400
49 0.049000 0.004900
21797 21.797000 2.179700
43 0.043000 0.004300
33993 33.993000 3.399300
85 0.085000 0.008500
```

Vscode

Chapter 4: Analysis and Comments

4.1.1 Algorithm 1: Brute-Force

Time Complexity
The brute-force algorithm examines every possible pair of distinct elements in the array. The outer loop runs Ntimes, and for each iteration of the outer loop, the inner loop runs approximately $N - i - 1$times. The total number of comparisons is:

$$\sum_{i=0}^{N-1}(N - i - 1) = \frac{N(N-1)}{2} = O(N^2)$$

Therefore, the time complexity is **O(N²)**. This quadratic growth makes the algorithm impractical for large input sizes (e.g., $N > 10^4$).

Space Complexity
The algorithm uses only a few integer variables for loop indices and temporary storage. No additional data structures are allocated that depend on the input size. Hence, the space complexity is **O(1)** (constant space).

4.1.2 Algorithm 2: Sort + Two Pointers

Time Complexity
The algorithm consists of two main phases:

1.Sorting: The array is sorted using the qsort function, which typically implements quicksort. Its average-case time complexity is **O(N log N)**. In the worst case (e.g., already sorted array with poor pivot selection), quicksort can degrade to **O(N²)**, but most C libraries use an optimized version (e.g., introsort) that guarantees **O(N log N)** worst-case performance.

2.Two-pointer search: After sorting, the two pointers traverse the array at most once, performing at most Nsteps. Each step involves a constant-time addition and comparison, so this phase is **O(N)**.

The overall time complexity is dominated by the sorting step, giving **O(N log N)**.

Space Complexity
The algorithm uses an extra array temp of size $N$to store a copy of the original data (to avoid modifying the input). This requires **O(N)** additional space. The sorting itself may require additional stack space for recursion (typically **O(log N)** in the average case for quicksort), but this is usually negligible compared to the auxiliary array. If we were allowed

to sort the original array in place, the space complexity could be reduced to **O(log N)** (for the recursion stack) or even **O(1)** for an iterative sort. However, in our implementation, we explicitly copy the array, so the space complexity is **O(N)**.

4.2 Comparison of Complexities

Algorithm	Time Complexity	Space Complexity	Suitable for
Brute-Force	$O(N^2)$	O(1)	Very small N (e.g., N < 5000)
Sort+Two-Ptr	O(N log N)	O(N)	Large N

The two-pointer algorithm is clearly superior for large datasets, offering a trade-off of extra memory for a dramatic speedup.

4.3 Further Possible Improvements

4.3.1 Handling Duplicate Elements

Both algorithms currently assume that the two numbers must come from distinct indices. If duplicates are allowed (e.g., using the same value from two different positions), the algorithms already work correctly as long as we ensure indices differ. However, special care is needed when the complement equals the current element: we must verify that there are at least two occurrences of that value. This can be handled by counting frequencies (e.g., using a hash map) or by slightly modifying the two-pointer logic to allow using the same value if it appears twice.

4.3.2 Early Termination in Brute-Force

If the array is sorted (or partially sorted), we could add a condition to break out of the inner loop early when the sum exceeds the target. However, sorting itself would add O(N log N) cost, defeating the purpose for small N. This improvement is only useful if the input is already sorted or if we are willing to sort for other reasons.

4.4 Conclusion

The brute-force algorithm is simple but only suitable for small datasets. The sorting-based two-pointer algorithm provides a good balance of performance and memory usage for most practical sizes. Further improvements can be made depending on the specific constraints (e.g., memory, multiple queries, duplicates). The choice of algorithm should be guided by the expected input size and available resources.

Appendix: Source Code (in C)

```
#include<stdio.h>
#include<time.h>
#include<stdlib.h>


//algorithm1:brute-force
//Time Complexity:  O(N squared)
//check all possible pairs of elements and use two
nested loops.
int algorithm1(int arr[],int N,int target,int *a,int
*b)
{
    for(int i =0;i<N;i++){ // outer loop
        for(int j=i+1;j<N;j++){ // inner loop
            if(arr[i]+arr[j]==target){ // judgement
                *a=arr[i];//store one addend
                *b=arr[j];//store another addend
                return 1;//1 represents we get what we
want.
            }
        }
    }
```

```
    return 0;//0 represents we failed to find a,b s.t.
a+b=target.
}
//the thought is little yet the consumption is
huge,especially on a large input.

//Necessary for the qsort function.
//Confirm the rule of compare.
int  my_compare(const void  *a,const void *b){ //
here most be const void ,it is a requirement of qsort
function
    return *(int *)a - *(int *)b;
}

//algorithm2:sort the array first,then search by two
pointers.
//Time Complexity O(NlogN)
//Sorting uses the qsort function by C standard
library
int algorithm2(int arr[],int N,int target,int
*a,int*b)
{
```

```
    qsort(arr,N,sizeof(int),my_compare);
    int left,right; // two indexs for searching.
    left=0;//initial element(the smallest one)
    right = N-1;// last element(also the largest one)
    while(left<right){//judging if it's repetitive.
        int sum= arr[left] + arr[right];//calculating
temp sum.
        if(sum == target){
            *a = arr[left];//store one addend
            *b = arr[right];//store another addend
            return 1;//representing a,b is found.
        }else if(sum<target){
        left++;  //make the small one larger.
        }else{
        right--; //make the big one smaller.
        }
    }
    return 0;//representing there isn't such a,b.
}

// Using this function to generate a random array
between 1 and V
```

```
// S is the array,N represents how many numbers are
in the array,V determines the range of the elements.
void generatearray(int S[],int N,int V){
            if(RAND_MAX>40000){
                for(int i =0;i<N;i++){
                    S[i]=rand() %V +1;
                }
            }//if the range of random number is big
enough,use it directly.
            else{
                for(int i =0;i<N;i++){
                    S[i]=((rand()<<15)|rand()) %V +1;
                }
            }//if not , we shall connect two random
number in order to produce what we want.
}

//Used for counting the time.
clock_t start,stop;
double duration;
```

```
void MeasureTime(int N,int T);//prototype,detailed
implement are behind.

int main(){
    srand(42);
    //set pseudo random seed.
    // as 42 is a certain number,everytime the program
run, we will get the same array.
    // but for a single time,we can treat it as random.
    //N is small ,run for several times
    MeasureTime(1000,1000);
    MeasureTime(5000,1000);
    MeasureTime(10000,1000);
    //N is huge,run for 10 times in a bid to eliminate
the affect of random number.
    MeasureTime(20000,10);
    MeasureTime(40000,10);
    MeasureTime(60000,10);
    MeasureTime(80000,10);
    MeasureTime(100000,10);
}
void MeasureTime(int N,int T){
```

```
    int  *s =(int *)malloc(sizeof(int)*N);
    int x,y;
    int *a,*b;
    a=&x,b=&y;//confirming pointer a ,b is legal.
    int total=0;//counting total ticks.
    double average,sum;//counting the average
running time and the total running time.
    sum=0;
    for(int i =1;i<=T;i++){//run for T times.
    generatearray(s,N,N);
    int target = rand()<<15|rand();
    start=clock();
    algorithm1(s,N,target,a,b);
    stop = clock();
    duration =
((double)(stop-start))/CLOCKS_PER_SEC;
    //The CLK_TCK is not used anymore,so here we use
    //CLOCKS_PER_SEC,which behaviours samely.
    sum+=duration;//counting the total time.
    total+=(int)(stop-start);//counting the total
ticks.
    }
```

```
    average = sum/T;//calculating the average time
for a single run.
    printf("%d %f %f\n",total,sum,average);//print
out the results.
    total =0;
    sum =0;
    average=0;
    //reset all these variables for algorithm2.
    for(int i =1;i<=T;i++){
    generatearray(s,N,N);
    int target = rand()<<15|rand();
    start=clock();
    algorithm2(s,N,target,a,b);
    stop = clock();
    duration =
((double)(stop-start))/CLOCKS_PER_SEC;
    sum+=duration;
    total+=(int)(stop-start);
    }
    average = sum/T;
    printf("%d %f %f\n",total,sum,average);
}
```